

IMPLEMENTATION GUIDE

Loop Engineering System Architecture

The Definitive Technical Framework and Copy-Pasteable Skeletons for
Building Autonomous, Self-Correcting AI Agents.

The 4 Primitives of an **AI Loop Harness**

Before wrapping an LLM in an autonomous loop wrapper, your systems infrastructure must provide four foundational modules:

-  **The Outer Loop Trigger:** Event-driven triggers (such as automated webhooks or cron workers tracking failing testing suites) that dispatch tasks to a standby execution runner completely unattended.
-  **The Isolated Tool Harness:** Strict containerized orchestration frameworks (like ephemeral Git worktrees, isolated Docker configurations, or MCP connectors) allowing secure, sandbox-bounded system tool access.
-  **The Independent Verifier:** Enforces an absolute separation of logical concerns. The agent instance that generated the file modifications must never act as the evaluator that grades its success.
-  **External Durable State:** Progress tracking and thread history mapped to database storage completely outside the model's runtime context window, preventing context-degradation over long loops.

System Prompt Template: Self-Correcting Execution

System Role & Runtime Environment

You are an autonomous engineering sub-agent operating within an isolated Git worktree. You have terminal access, a compiler, and a local test runner.

Objective & Exit Conditions

Your task is complete ONLY when all conditions inside the <goal_spec> pass cleanly. If an action results in an error, you must parse the trace, deduce the flaw, and try a modified approach.

The Loop Protocol

For each iteration step, you must execute exactly in this sequence:

1. ****Perceive & Reason:**** Analyze current codebase state. Detail observations inside <thinking> tags.
2. ****Act:**** Invoke the single most precise tool required (e.g., file edit, test run).
3. ****Observe:**** Read the execution or compiler feedback. If errors exist, loop immediately back to step 1.

Managing Runtime Hazards: The Circuit Breaker

Unmonitored agentic runs introduce systemic infrastructure risks. Your system wrapper must contain rigid bounds against these 3 loops:

-  **Infinite Stagnation Loops (Token Burners):** Occurs when a model continuously hits the identical compiler trace and repeats the exact same code approach indefinitely. Mitigate this by deploying a circuit breaker that aborts after 3 identical states.
-  **Context Window Degradation:** As multi-step agent actions run for hours, tool history strings quickly flood token capacity, causing reasoning drop-off. Programmatically prune old tool results out of the active context history.
-  **Cognitive Surrender:** The systemic trend where engineering teams become completely passive consumers of AI commits. Maintain mandatory peer reviews at the PR merge boundary.